

Iteration 3

Step 2 - Establish the Iteration Goal

The goal of this iteration is to address architectural concerns in regards to performance, security, and system observability, ensuring the architecture will both safely and reliably support the primary operations of the system. This iteration is focused on refinement of cross-cutting structures that were only addressed partially in previous iterations.

Key quality attributes, and concerns that are guiding this iteration include:

QA-1: Performance - The system should be able to provide quick responses for academic and dashboard data that is commonly accessed.

QA-2: Availability - The system should have the ability to be monitored effectively, and keep operating through fault detection.

QA-4: Security - Access to sensitive academic information should be consistently protected.

Supporting Drivers

- CON-1: University privacy rules surrounding sensitive academic information
- CRN-3: Protection, and the proper handling of academic information
- CRN-5: Real-time health and system metric access

Step 3 - Choose One or More Layers to Refine

The elements that we will refine in this iteration are modules located in the different layers defined in previous iterations. The selected quality attributes being performance, availability, and security, all globally affect the system causing this iteration to have a focus on refinement of cross cutting components instead of functionality isolated to one layer.

Step 4 - Choose One or More Design Concepts that Satisfy the Selected Drivers

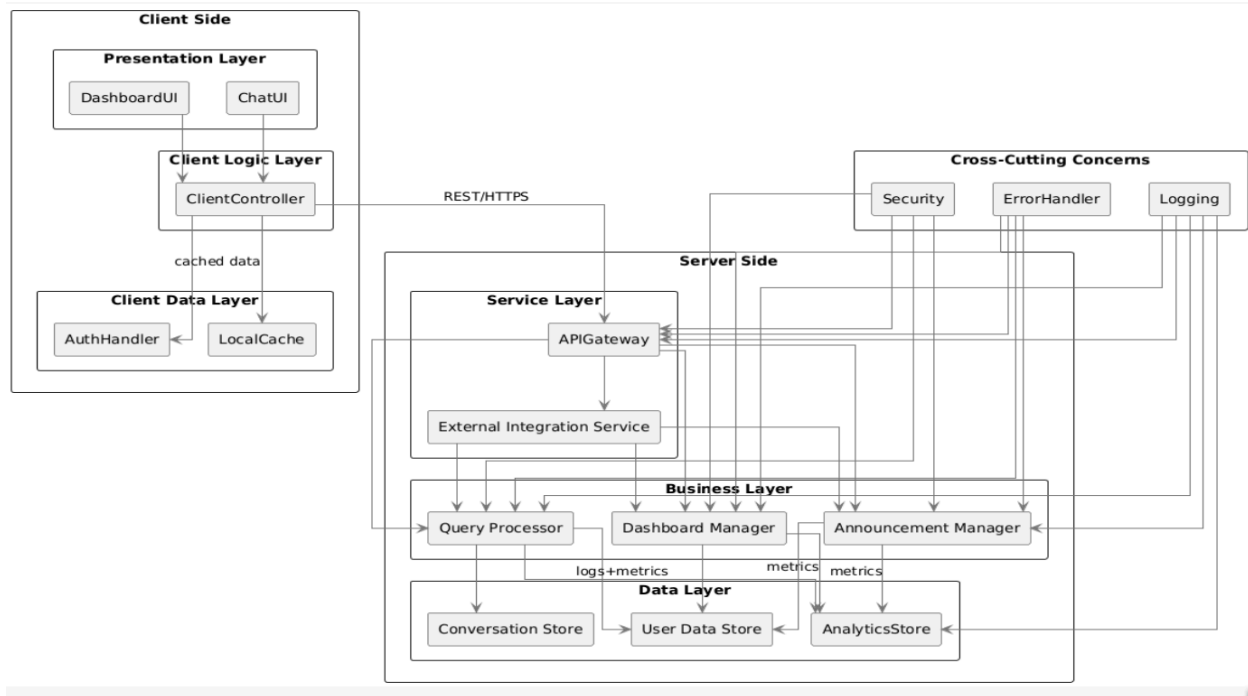
Design Decision and Location	Rationale
Introduce Centralized Security Enforcement Component (Cross-Cutting Layer)	With security being centralized, a single mechanism can validate all access to sensitive academic data. This directly supports QA-4 (Security), CON-1 (University-wide privacy rules), and CRN-3 (Secure handling of academic information) through prevention of duplicated or inconsistent permission checks across modules.
Implement Unified Logging Pipeline across every module (Cross-Cutting Layer)	A shared logging structure captures key actions in a format that is consistent (queries, model updates, announcements). This directly supports UC-7 as well as QA-2 through improvement of monitoring of performance issues, failures, and integration errors, It also sets the architecture up for more monitoring and analytics refinements.
Define a system-wide strategy for Error Handling	Consistent error handling approach leads to predictable behaviour for clients and maintainers by ensuring all modules report failures in the same format. This directly supports CRN-5 as it allows for easier

	tracking of system health, and reduces complexity of debugging during operational issues.
Introduce Local Caching Rules for the Client Data Layer	Unnecessary server requests are reduced by specification of when Local Cache must return saved academic or dashboard data. This directly supports QA-1 (performance) via reduction of latency for information that is frequently accessed, and providing more resilience in limited connectivity scenarios.
Define Metrics Collection in the Analytics Store	Extension of the Analytics Store to collect latency, error rates, and request counts directly supports UC-7, and CRN-5. This further improves system health visibility, also supporting QA-2 (availability) by enabling fast outage detection, and performance degradation.

Step 5 - Instantiate Architectural Elements, Allocate Responsibilities, Define Interfaces

Design Decision	Rationale
Introduce Centralized Security Enforcement Component (Cross-Cutting Layer)	The Centralized Security Enforcement Component enforces token validation, role-based access checks, and privacy rules for any sensitive academic operation. Thus, it becomes the single port of enforcement used by every service in order to maintain consistent security behaviour across the whole system.
Implement Unified Logging Pipeline across every module (Cross-Cutting Layer)	The logging pipeline provides a standard for how modules emit logs, giving a unified structure for recording queries, model updates, announcements, and integration events. This supporting operational monitoring, and simplifying analysis that may arise later.
Define a system-wide strategy for Error Handling	This strategy defines error objects that are common, return formats, and rules of escalation. It helps ensure failures that occur across modules are consistently handled, and that error information is both clear and traceable for maintainers.
Introduce Local Caching Rules for the Client Data Layer	Local Caching Rules specify when either cached academic or dashboard data should be retrieved from the server, or to be returned locally. This improves the response time for information that is frequently accessed, and reduces overall server load.
Define Metrics Collection in the Analytics Store	The Analytics Store is extended to collect latency, error rate, as well as request count metrics. The metrics with real time system health insight, as well as support availability monitoring.

Step 6 - Sketch views and Record Design Decisions



Step 7 - Perform Analysis of Current Design and Review Iteration

Driver	Not Addressed	Partially Addressed	Completely Addressed	Design Decisions
QA-1 (Performance)		✓		Introduction of Local Caching rules significantly reduces unnecessary server requests and improves response times for frequently accessed academic and dashboard data. However, since deeper performance tuning is yet to be finalized as such the driver is partially addressed.
QA-2 (Availability / Observability)		✓		The Unified Logging Pipeline and Metrics Collection in the Analytics Store improve system health visibility, fault detection and outage identification. Although monitoring technologies are not yet chosen, this driver is partially addressed.
QA-4 (Security)		✓		The centralized security enforcement component creates a

				single enforcement layer for authentication, authorization and privacy rules. This greatly reduces the risk of inconsistency and supports protection of academic data. However, Technology selection and full threat modelling have yet to be implemented, as such the driver is partially addressed.
CON-1 (University privacy rules)		✓		The centralized security components ensures a consistent enforcement of privacy rules across all modules. Additional validation processes are still to be designed
CRN-3 (Protection and handling of academic information)		✓		Addressed through centralized security and controlled data-access pathways. Final encryption and secure transport specifications yet to be selected.
CRN-5 (Real-time health and system metric access)		✓		Metrics collection and unified logging enable health monitoring, error visibility, and real time performance tracking. Full monitoring stack integration is yet to be implemented
CON-4	✓			No relevant decision made during this iteration
CON-5	✓			No relevant decision made during this iteration
CON-6	✓			No relevant decision made during this iteration
CRN-2	✓			No relevant decision made during this iteration
CRN-4	✓			No relevant decision made during this iteration

Category	Item	Description
Architectural Drivers	QA-1 Performance	The system should support fast retrieval of common dashboard & academic data
	QA-2 Availability/ Observability	System should detect faults, monitor performance, and remain operational through failures
	QA-3 Reliability/ Fault Tolerance	System must remain operation through server failures and provide redundancy to avoid downtime
	QA-4 Security	Sensitivity academic information must be protected through consistent access control and privacy rules
	CON-1	University privacy rules must be respected in all storage and transfer of academic data
	CRN-3	Academic Information must be properly secured
	CRN-5	Real-time system health, logs and metrics must be available for administrators
Architecture Approaches	Seperate replicated trap receiver/ redundancy	Introduced in earlier iteration to reduce failure probability and improve reliability.
	Redundant application servers with load balancer	Supports fault tolerance and reliability.
	Centralized security Enforcement Component	Enforces consistent rules for authentication, authorization, and privacy.
	Unifiied Logging Pipeline	Provides uniform event capture across all modules to support availability /observability
	System-wide Error Handling Strategy	Standardizes error formats and reporting to improve diagnoability

	Local Caching Rules in Client Data Layer	Reduces request load and improves performance of frequent queries
	Metrics Collection in Analytics Store	Tracks latency, error rates, and request counts for fault detection
Sensitivity Points	Caching Rules Affect Performance and Freshness	Aggressive caching improves performance, but risks returning outdated academic data
	Centralized Security Component Latency	Every sensitive request goes through the same security gateway, improving security but potentially adding response time overhead.
	Unified Logging Volume	Heavy logging can introduce system overhead and affect performance if not rate-limited
	Metrics Collection Overhead	Real-time metric capture improves availability but may increase CPU/IO utilization
Trade-off Points	Security vs. Performance	Centralized security strengthens protection, but may slow high-traffic or large-volume data access
	Caching vs. Data Freshness	Local Caching reduces latency but risks returning stale academic entries.
	Logging Detail vs. Runtime Cost	Detailed logs support observability but may degrade performance
	Redundancy vs. Cost/Complexity	Replication and load balancing increase fault tolerance
Risks	Security Implementation Not Yet Technology-Specific	Lack of chosen technologies leaves security partially addressed
	Logging Backend Not Defined	Logging pipeline is designed but not mapped to a concrete log-aggregation technology

	Metrics Framework Not Selected	Metrics collection plan exists, but no decision on Prometheus, OpenTelemetry etc
	Cache Inconsistency Risk	Without strong cache invalidation rules, clients may see outdated academic data
	Observability Gaps	Metrics and logs exist, but without dashboards, operators may still lack timely insight
Non-Risks (Confirmed Strengths)	Uniform Security Enforcement	Centralized approach eliminates inconsistent access control across modules
	Improved Fault Detection	Logging + metrics give good system visibility, even without complete tooling choices
	Improved Performance for Frequent Access	Caching significantly reduces load and latency for commonly accessed dashboard data
	Consistent Error Handling	System-wide strategy ensures failures behave in a predictable manner